

Experiment 6: Time Series Analysis of Website Traffic

Aim:

To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

Description:

The website traffic dataset contains daily data for pageviews, unique visitors and bounce rate. The data is loaded and cleaned by converting the data column into a datetime format. Time series decomposition is used to identify trend, seasonal and residual components. A moving average is used to observe overall trend, and the z-score method is used to detect anomalies in page views.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
uploaded = files.upload()
data = pd.read_csv("website_traffic_timeseries.csv")
data["Date"] = pd.to_datetime(data["Date"])
data.set_index("Date", inplace=True)
data.interpolate(method="linear", inplace=True)
print(data.head())
print(data.info())
print(data.describe())
plt.figure(figsize=(14, 6))

plt.plot(data.index, data["Page_Views"], label="Page Views")
```

```
plt.plot(data.index, data["Unique_Visitors"], label="Unique Visitors")
```

```
plt.plot(data.index, data["Bounce_Rate"], label="Bounce Rate")
```

```
plt.title("Website Traffic Over Time")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Values")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
decomposition = seasonal_decompose(
```

```
    data['Page_Views'],
```

```
    model='additive',
```

```
    period=7
```

```
)
```

```
decomposition.plot()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()
```

```
# Plot original data and moving average
```

```
plt.figure(figsize=(14, 6))
```

```
plt.plot(data.index, data["Page_Views"], label="Page Views")
```

```
plt.plot(data.index, data["Page_Views_MA"], label="30-Day Moving Average")
```

```
plt.title("Page Views with Moving Average")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Page Views")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
data["Page_Views_Z"] = (  
    (data["Page_Views"] - data["Page_Views"].mean())  
    / data["Page_Views"].std()  
)
```

```
# Identify anomalies
```

```
anomalies = data[data["Page_Views_Z"].abs() > 3]
```

```
print("Detected Anomalies:")
```

```
print(anomalies[["Page_Views", "Page_Views_Z"]])
```

```
plt.figure(figsize=(14, 6))
```

```
plt.plot(  
    data.index,  
    data["Page_Views"],  
    label="Page Views"  
)
```

```
# Highlight anomalies

plt.scatter(
    anomalies.index,
    anomalies["Page_Views"],
    color="red",
    label="Anomalies"
)

plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.grid(True)

plt.show()

import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Load dataset

data = pd.read_csv("website_traffic_timeseries.csv", parse_dates=["Date"],
index_col="Date")

# Handle missing values

data.interpolate(method="linear", inplace=True)
```

```
# Display first few rows
```

```
print(data.head())
```

```
# Dataset information and summary
```

```
print(data.info())
```

```
print(data.describe())
```

```
# Time Series Decomposition
```

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
decomposition = seasonal_decompose(
```

```
    data["Page_Views"],
```

```
    model="additive",
```

```
    period=7
```

```
)
```

```
decomposition.plot()
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Moving Average
```

```
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()
```

```
data[["Page_Views", "Page_Views_MA"]].plot(figsize=(12, 5))
```

```
plt.title("Page Views with Moving Average")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Page Views")
```

```
plt.show()
```

```
# Anomaly Detection using Z-score
```

```
data["Page_Views_Z"] = (
```

```
    (data["Page_Views"] - data["Page_Views"].mean())
```

```
    / data["Page_Views"].std()
```

```
)
```

```
anomalies = data[data["Page_Views_Z"].abs() > 3]
```

```
print("Anomalies:")
```

```
print(anomalies[["Page_Views", "Page_Views_Z"]])
```

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(data.index, data["Page_Views"], label="Page Views")
```

```
plt.scatter(
```

```
    anomalies.index,
```

```
    anomalies["Page_Views"],
```

```
    color="red",
```

```
    label="Anomalies"
```

```
)
```

```
plt.title("Website Traffic Anomaly Detection")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Page Views")
```

```
plt.legend()
```

```
plt.show()
```

Output:

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving website_traffic_timeseries.csv to website_traffic_timeseries.csv

```
Page_Views Unique_Visitors Bounce_Rate
```

```
Date
```

2025-01-01	2074	1280	48.81
2025-01-02	2326	1306	44.98
2025-01-03	2999	1678	45.09
2025-01-04	3185	1863	45.93
2025-01-05	2586	1555	50.24

```
<class 'pandas.core.frame.DataFrame'>
```

DatetimeIndex: 365 entries, 2025-01-01 to 2025-12-31

Data columns (total 3 columns):

```
# Column Non-Null Count Dtype
```

```
--- -----
```

```
0 Page_Views 365 non-null int64
```

```
1 Unique_Visitors 365 non-null int64
```

```
2 Bounce_Rate 365 non-null float64
```

dtypes: float64(1), int64(2)

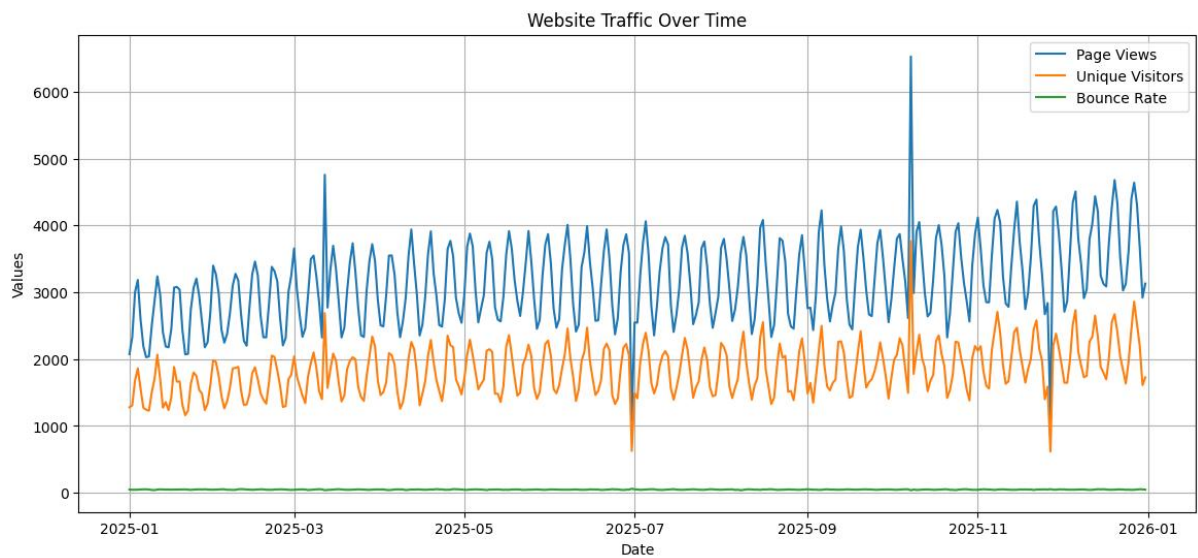
memory usage: 11.4 KB

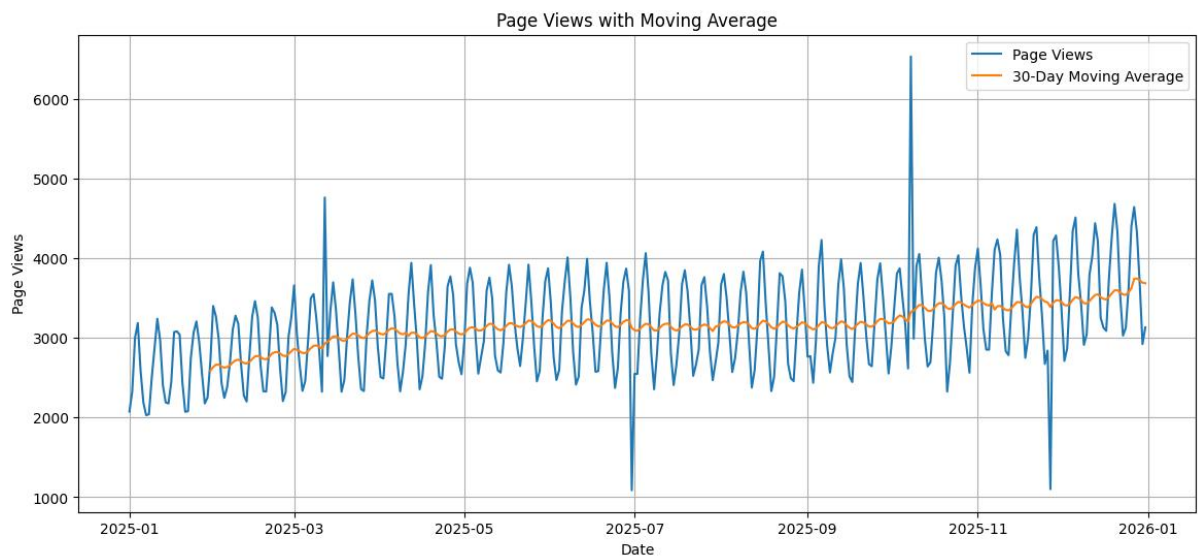
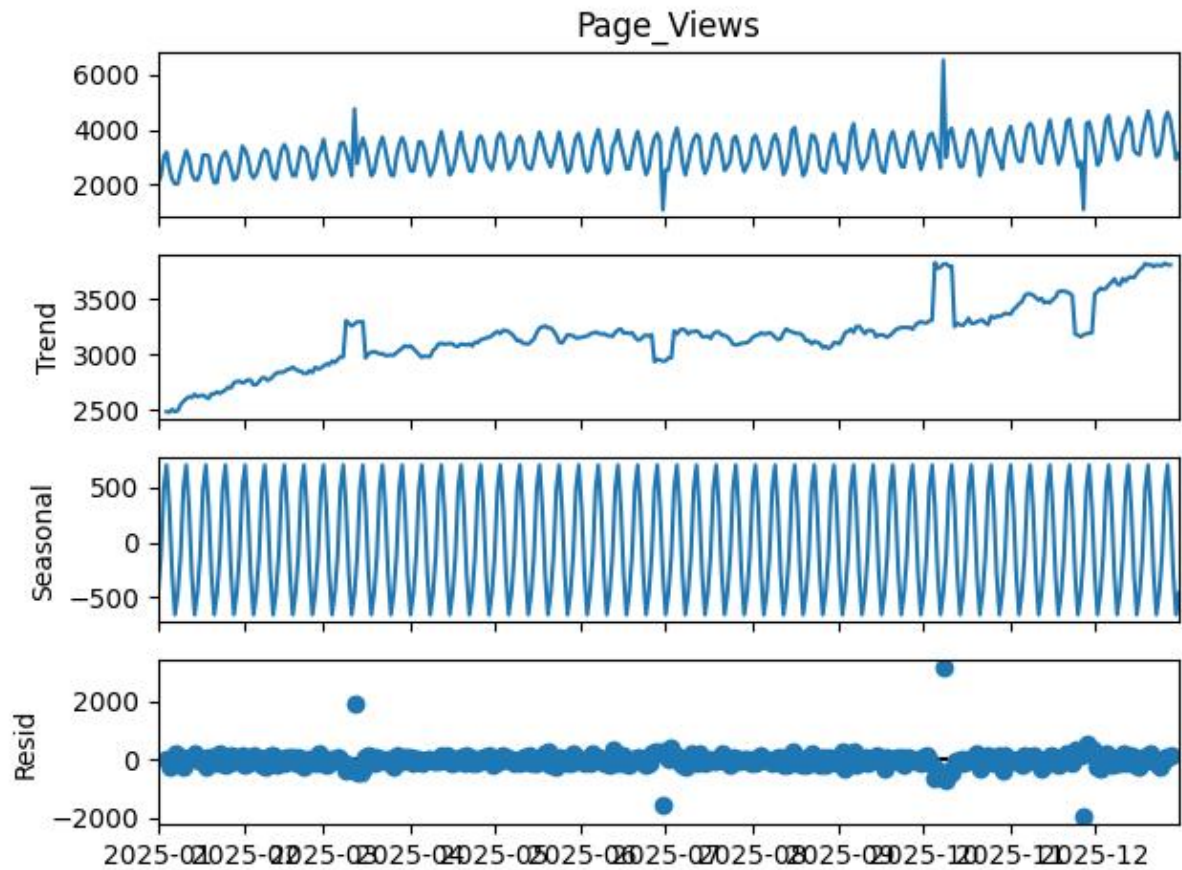
None

```
Page_Views Unique_Visitors Bounce_Rate
```

```
count 365.000000 365.000000 365.000000
```

mean	3166.630137	1830.115068	48.036904
std	623.902005	371.898955	3.479127
min	1087.000000	617.000000	37.530000
25%	2686.000000	1550.000000	45.310000
50%	3105.000000	1797.000000	48.240000
75%	3614.000000	2083.000000	50.680000
max	6529.000000	3764.000000	60.550000

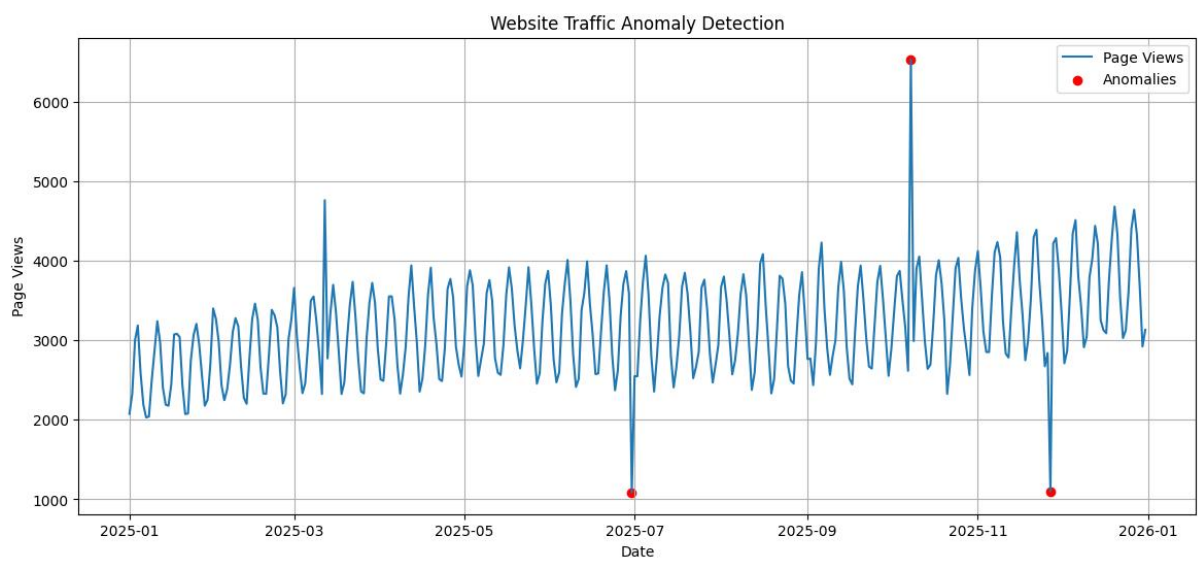




Detected Anomalies:

	Page_Views	Page_Views_Z
2025-06-30	1087	-3.333264
2025-10-08	6529	5.389260

2025-11-27 1101 -3.310825



Page_Views Unique_Visitors Bounce_Rate

Date			
2025-01-01	2074	1280	48.81
2025-01-02	2326	1306	44.98
2025-01-03	2999	1678	45.09
2025-01-04	3185	1863	45.93
2025-01-05	2586	1555	50.24

<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 365 entries, 2025-01-01 to 2025-12-31
Data columns (total 3 columns):

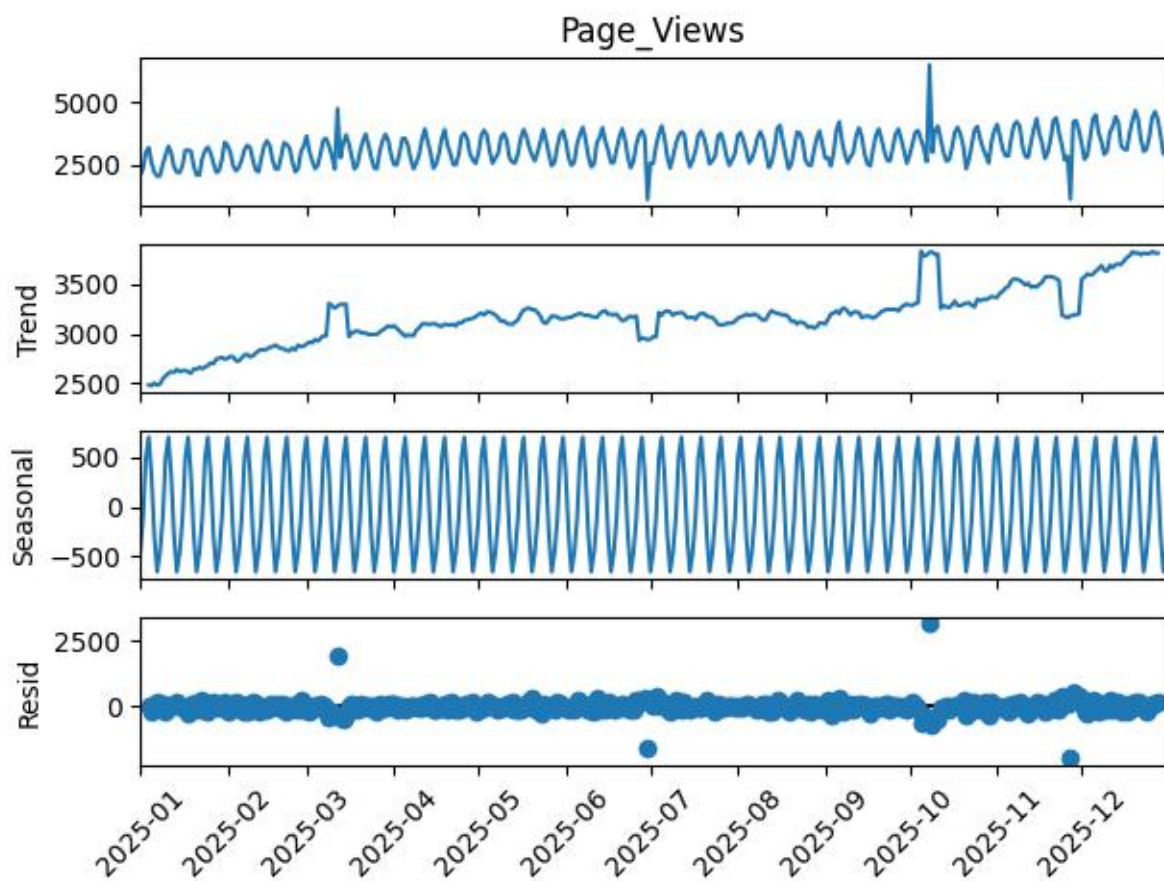
#	Column	Non-Null Count	Dtype
0	Page_Views	365 non-null	int64
1	Unique_Visitors	365 non-null	int64
2	Bounce_Rate	365 non-null	float64

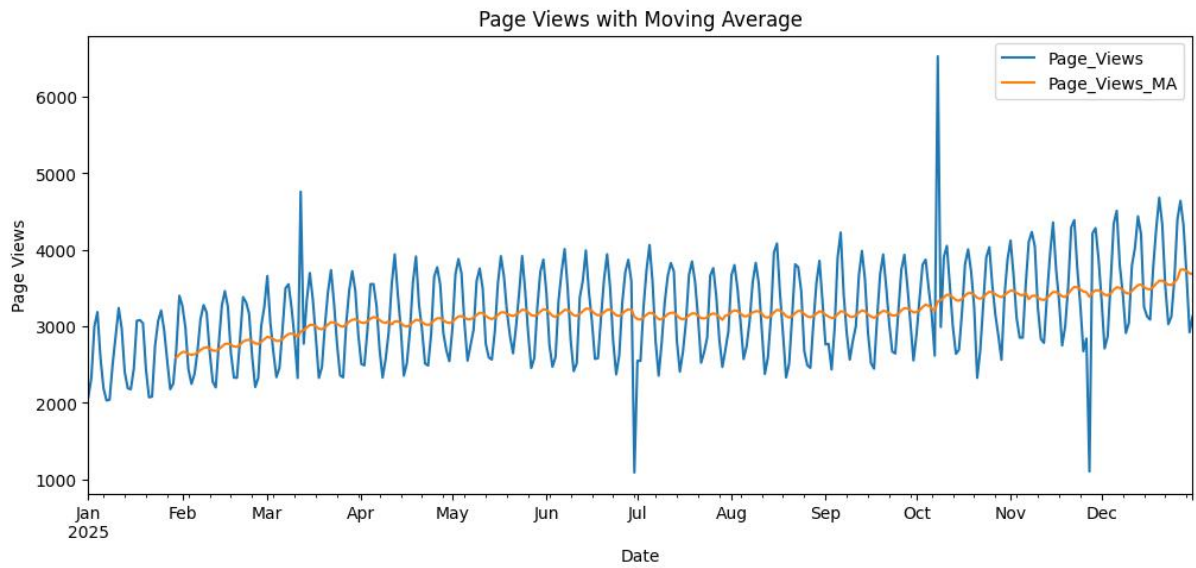
dtypes: float64(1), int64(2)

memory usage: 11.4 KB

None

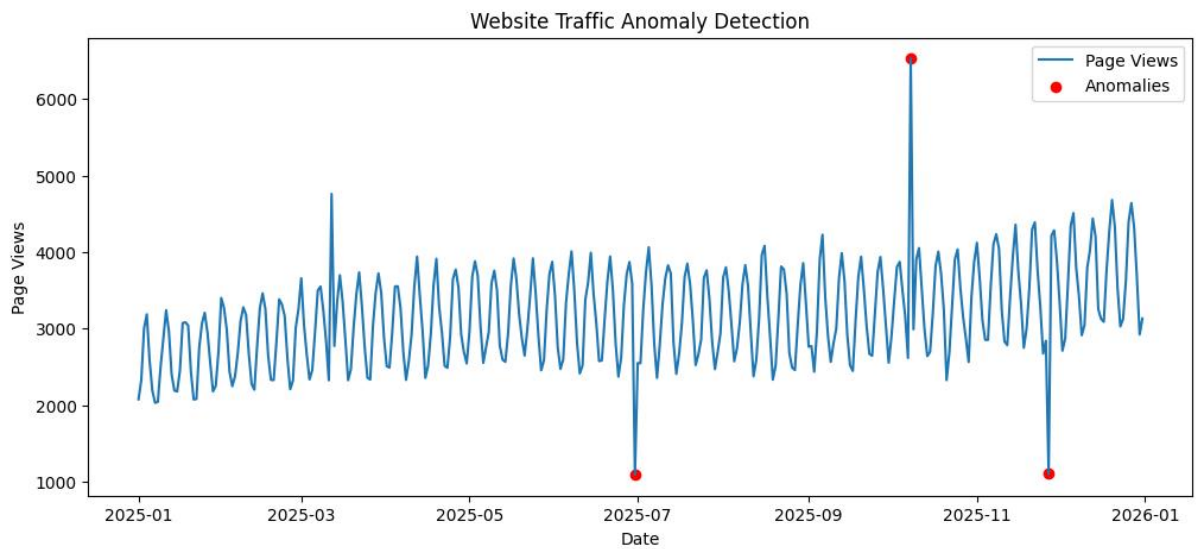
	Page_Views	Unique_Visitors	Bounce_Rate
count	365.000000	365.000000	365.000000
mean	3166.630137	1830.115068	48.036904
std	623.902005	371.898955	3.479127
min	1087.000000	617.000000	37.530000
25%	2686.000000	1550.000000	45.310000
50%	3105.000000	1797.000000	48.240000
75%	3614.000000	2083.000000	50.680000
max	6529.000000	3764.000000	60.550000





Anomalies:

	Page_Views	Page_Views_Z
Date		
2025-06-30	1087	-3.333264
2025-10-08	6529	5.389260
2025-11-27	1101	-3.310825



Result:

The website traffic over time is visualized using a line plot, showing the variations and sudden increases in page view.